UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

FUNDAMENTOS DE PROGRAMACIÓN

Nombre: Luis Alexander Quishpe Mendoza Fecha: 3/11/2026

NRC: 36466

Seudocódigo y Código con Buenas PRACTICAS - - -

1.1 Media de dos Valores

Seudocódiso

```
ALGORITMO MediaDeDosValores

    Definir num1, num2, media Como Real
    Escribir "Ingrese el primer número:"
    Leer num1
    Escribir "Ingrese el segundo número:"
    Leer num2
    media <- (num1 + num2) / 2
    Escribir "La media es:", media

FinALGORITMO
```

Código

```
#include <Studio.h>
int main()
{
   double num1, num2, media; // declara números con decimales (3.5, 4.5)

   Printf ("Ingrese el Primer número: "); // Pide el primer dato ingresado por teclado
   Scanf ("%.1f , &num1); // Se guarda el dato

   Printf ("Ingrese el segundo número: "); // Pide segundo número por teclado
   Scanf("%1f, &num2); // Se guarda el dato

   media = (num1 + num2) / 2; // Calculamos la media, los parentesis van primero y
                              después se opera la división para 2
   Printf ("la media es: %2f\n", media); // Resultado
}
```

1.2. Valor absoluto de x al cubo

Seudocódigo

```
ALGORITMO ValorAbsoluto

    Definir x, valorAbs, resultado como Real
    Escribir "Ingrese un número"
    Leer x
    valorAbs ← abs(x)
    resultado ← valorAbs ^ 3
    Escribir "El valor de |x|^3 es: ", resultado

Fin ALGORITMO
```

Código

```
# include <stdio.h>
int main ()
{
    double x, valorAbs, resultado; // Definimos las variables

    Printf ("Ingrese un número: "); // pide el dato
    scanf ("%.1f", &x);                    // se guarda

    if (x <0)                      // Si x es negativo se cambia de signo
        valorAbs = -x;
    else
        valorAbs = x;              // Si no queda tal cual.

    resultado = valorAbs *valorAbs * valorAbs; // El valor elevado al cubo

    Printf("El valor de |x|^3 es: %.2f\n", resultado);

}
```

1.3 Divisible

Seudocódigo

```
ALGORITMO Divisible

    Definir x, y como Entero
    Escribir "Ingrese el primer número entero: "
    Leer x
    Escribir "Ingrese el segundo número entero: "
    Leer y

    Si y = 0 Entonces
        Escribir x " no es posible dividir por ", y
    SINO
        Escribir x " [illegible]
```

```
        Si x MOD y =0 Entonces
            Escribir x, " es divisible por ", y
        SiNo
            Escribir x, " no es divisible por ", y
        FinSi
    FinSi

FIN ALGORITMO
```

Código

```
#include <stdio.h>
  int main ()
{
    int x, y;          // declaracion como enteros

    Printf("Primero número:");      // Pide el dividendo
    scanf("%d", &x);                // Lee el dividendo
    Printf("Segundo número: ");     // Pide el divisor
    scanf("%d", &y);                // Lee el divisor

    if (y == 0)                     // Valida antes de dividir
  {
    Printf("No es posible dividir entre cero\n");  // Aviso en vez de error
  }
    else                            // Siempre y cuando no sea cero
  {
        if (x %. y==0)
            Printf("%.d es divisible por %.d\n", x, y);  //Cada %.d toma una variable.
        else
            Printf("%.d no es divisible por %.d\n", x, y); // El resto no es 0
  }
    return 0;                       // Tiene un fin correcto
}
```

1.4 Intervalo

Seudocódigo

```
ALGORITMO Intervalo

    Definir x Como Real
    Escribir "Ingrese un numero: "
    Leer x

    Si x >=0 Y x <= 10 Entonces
        Escribir x, "Pertenece al intervalo [0, 10]"
    SiNo
        Escribir x, "no pertenece al intervalo [0, 10]"
    FinSi
FIN ALGORITMO
```

Código

```
#include <studio.h>
 int main ()
{
  double x;                    // variable que acepta decimales

  Printf (" Ingrese un número:  ");   //Pide el dato
  Scanf ("%.1f", &x);                 // Lee el dato

  if (x >=0 && x <=10)         // Condición doble que deben cumplirse
     Printf("%.2f Pertenece al intervalo [0, 10]\n", x);
  else                                // Si falla al menos una condición
     Printf("%.2f no pertenece al intervalo [0, 10]\n", x);
  return 0;                    // Fin correcto
}
```

1.5 Conversión de Unidades de tiempo

Seudocódigo

```
ALGORITMO Conversion_Tiempo

   Definir C, dias, horas, minutos, segundos como Entero
   Escribir "Ingrese la duración en segundos: "
   Leer c

   Si c < 0 Entonces
      Escribir "La duración no puede ser negativa"
   Sino
      dias <- trunc (c / (24*60*60))
      horas <- trunc ((c MOD (24*60*60)) / (60*60))
      Minutos <- trunc ((c MOD (60*60)) / 60)
      Segundos <- c MOD 60
      Escribir C, " Segundos equivalentes a ", dias, horas, "horas", Minutos,
      "Minutos", y , Segundos, "Segundos"
   FinSi

FIN ALGORITMO
```

Código

```
#include <studio.h>
 int main ()
{
   int c, dias, horas, minutos, segundos;   // Definimos variables como Enteras

  Printf ("Ingrese la duración en segundos: ");   // Pide el dato
  scanf ("%d", &c);                               // Lee los datos

  if (c <0)                    // condición de que una duración no puede ser
  {                               negativo.
```

```
        Pirintf("La duración no puede ser negativa\n");  // Aviso
}
else                                    // Dato válido
{
    dias = c/(24*60*60);          // Días completos, sin trunc
    horas = (c%(24*60*60))/(60*60); // % Deja lo que sobra de los días
    Minutos = (c%(60*60))/60;     // % deja lo que sobra de las horas
    segundos = c%60;              // Lo que sobra de los minutos

    Printf("%d segundos equivalen a %d días, %d horas, %d minutos y %d
    segundos\n", c, dias, horas, minutos, segundos);
}
return 0;
}
```

1.6 Máximo y Mínimo y Media de Alturas.

Seudocódigo

```
ALGORITMO Alturas_max_min

    Definir altura, suma, media como Real
    Definir máximo, mínimo como Real
    Definir cantidad como Entero

    Suma <- 0
    Cantidad <- 0

    Escribir "Altura (negativo para salir):"
    Leer altura

    Si altura >=0 Entonces
        maximo <- altura
        minimo <- altura
        Mientras altura >=0 Hacer
            suma <- suma + altura
            Cantidad <- Cantidad + 1
            Si altura > maximo Entonces
                maximo <- altura
            FinSi
            Si altura < minimo Entonces
                minimo <- altura
            FinSi
            Escribir "otra altura; "
            Leer altura
    FinMientras

    media <- suma / Cantidad
    Escribir "Media: ", media
    Escribir "Maximo: ", maximo
    Escribir "Minimo: ", minimo
```

ESTILO

SINO
    Escribir "No hay alturas válidas"
FinSi

Fin ALGORITMO

Código

```
# include <stdio.h>
 int main ()
{
    double altura, suma = 0, maximo, minimo, media; // declaración de variables
    int cantidad = 0;

    Printf("Altura (negativa para salir): "); // Pide primera altura
    scanf("%lf", &altura);                    // Lectura de la altura

    if (altura >=0)                  // Si la primera ya es negativa, no hay nada que evaluar
    {
        maximo = altura;  // La primera altura es el max inicial y el minimo
        minimo = altura;     inicial

        while (altura >=0)  // Repite mientras no sea negativa
        {
            suma = suma + altura    // Acumulador: suma la altura actual
            cantidad ++;            // Contador

            if (altura > maximo)    // Si supera el máximo actual
                maximo = altura;    // Es el nuevo máximo
            if (altura > minimo)    // Si es menor que el min actual
                minimo = altura;    // Es el nuevo mínimo

            Printf("Otra altura: ");  // Pide siguiente número
            scanf("%lf", &altura);    // Lee
    }

    Media = suma / cantidad;  // Seguro: aquí cantidad es al menos 1
    Printf("Media: %.2f\n", media);  // Se muestra la media
    Printf("Maximo: %.2f\n", maximo);  // Se muestra el máximo
    Printf("Minimo: %.2f\n", minimo);  // Se muestras mínimo
    }
    else                        // Primera altura negativa
    {
        PrintfC("No hay alturas validas\n");
    }
    return 0;                   // Fin Correcto
}
```

1.7 Raiz Cuadrada entera

Subcódigo

```
ALGORITMO  Raiz_Cuadrada_Entera

    Definir N, i como Entero
    Escribir "Ingrese un número entero positivo: "
    Leer N

    Si N <= 0 Entonces
        Escribir "El número debe ser positivo"
    SiNo
        i <- 1
        Mientras  i * i <= N  Hacer
            i <- i + 1
        FinMientras
        Escribir "La raiz cuadrada entera de ", N, " es ", i - 1
    FinSi

FinALGORITMO
```

Código

```
#include <stdio.h>
int main ()
{
   int N, i;                    // Definimos variables como enteros

   Printf("Ingrese un entero positivo:")  // Pide el dato
   scanf("%d", &N);                       // Lee el dato

   if (N <=0)                   // Condicion: Pide N positivo
   {
      PrintfC"El numero debe ser positivo\n")  // Aviso
   }
   ebe                          // Si el dato es valido
   {
      i = 1;                    // Primer dato (R >0)
      while (i * i <=N)  // <= Porque si i*i es igual a N ese i es ruiz.
      {
         i++;                   // Prueba el siguiente: garantiza el ciclo
      }

      PrintfC"La raiz cuadrada entera de %d es %d\n", N, i-1);

   }

   return 0;                    // fin Correcto

}
```

ESTILO

1.8 Número Primo

Seudocódigo

```
ALGORITMO Número Primo

    Definir N, i Como Entero
    Definir esPrimo Como Lógico
    Escribir "Inserte un número entero mayor que 1: "
    Leer N

    Si N <= 1 Entonces
        Escribir "El número debe ser mayor que 1"
    Sino
        esPrimo <- Verdadero
        i <- 2
        Mientras i < N Y esPrimo Hacer
            Si N MOD i = 0 Entonces
                esPrimo <- Falso
            Fin Si
            i <- i + 1
        Fin Mientras

        Si esPrimo Entonces
            Escribir N, " es primo"
        Sino
            Escribir N, " no es primo"
        Fin Si
    Fin Si

Fin ALGORITMO
```

Código

```
#include <stdio.h>
int main()
{
    int N, i, esPrimo;          // Declaración de variables

    Printf("Inserte un entero mayor que 1: ");   // Pide dato
    scanf("%d", &N);                             // Lee dato

    if (N <= 1)                 // Validar: 1, 0 y negativos no son primos.
    {
        Printf("El número debe ser mayor que 1\n");
    }
    else
    {
        esPrimo = 1;            // Se supone primo hasta que se demuestre lo contrario
        i = 2;                  // Primera divisor útil (1 / todos).
```

```
While (i<N && esPrimo)          // Sigue Mientras haya candidatos y No se haya
{                                  divisor
    if (N % i == 0)             // ¿i divide exactamente N?
        esPrimo = 0;            // Entonces No es primo
    i++;                        // Siguiente dato.
}
if (esPrimo)                    // Decide el mensaje mostrado
    Printf("%d es primo\n", N);
else
    Printf("%d no es primo\n", N);
}
    return 0;                   // Fin Correcto
}
```

1.9 Primos entre dos números

```
Funcion primo <- Es_primo (num)
    Definir primo como Logico
    Definir d como entero
    Si num <= 1 Entonces
        primo <- Falso
    Sino
        primo <- Verdadero
        Para d <- 2 Hasta trunc (raiz (num)) Hacer
            Si num Mod d = 0 Entonces
                primo <- Falso
            FinSi
        FinPara
    FinSi
Fin Función

ALGORITMO Primo_Entre_dos_Numeros
    Definir inicio, fin_rango, p como Entero
    Escribir "Inicio del Rango:"
    Leer Inicio
    Escribir "Fin del rango: "
    Leer fin_rango
    si inicio > fin_rango Entonces
        Escribir "error"
    Sino
        Escribir "Primos del rango:"
        Para p <- inicio Hasta fin_rango Hacer
            si Es_Primo (p) Entonces
                Escribir sin Saltar p, " "
            FinSi
        FinPara
    FinSi

FIN ALGORITMO
```

ESTILO

Código

```
#include <stdio.h>
int Es_Primo (int num)
{
    int primo, d;                    // variables locales: inician y finalizan con cada llamada
    if (num <=1)                     // 1, 0 y negativos no son primos
    {
        primo =0;
    }
    else
    {
        primo=1;                     // se supone primo
        for (d=2; d*d <=num; d++) // d*d <= num equivale a llegar hasta la raiz
        {
            if (num % d ==0) // ¿d divide a num?
                primo =0; // Entonces no es primo
        }
    }
    return primo;
}
int main ()                          // Módulo principal
{
    int inicio, fin_rango, p;

    Printf("Inicio del rango: ");    // Pide primer extremo
    Scanf("%d", &inicio);            // lo guardamos en inicio
    Printf("Fin del rango: ");       // Pide segundo extremo
    Scanf("%d", &fin_rango);         // lo guardamos en fin_rango

    if (inicio > fin_rango)          // validar el orden del rango
    {
        Printf("Error \n");          // Aviso
    }
    else
    {
        Printf("Primos del rango: "); // Encabezado de la lista
        for (p= inicio; p<= fin_rango; p++) // Recorre cada # (extremos incluidos)
        {
            if (Es_Primo(p))         // Llama al Módulo; ¿es V (No hace falta ==)
                Printf("%d," p);
        }
        Printf("\n");                // Cierra la línea una sola vez
    }
    return 0;                        // fin Correcto
}
```

1.10 Factorial

Seudocódigo

ALGORITMO Factorial

    Definir n, i, factorial Como Entero
    Escribir "Ingrese un entero entre 0 y 12: "
    Leer n
    Si n < 0 Entonces
        Escribir "El factorial no está def. para negativos"
    SiNO
        Si n > 12 Entonces
            Escribir "Solo se calcula hasta 12!"
        SiNO
            Factorial <- 1
            Para i <- 1 Hasta n Con Paso 1 Hacer
                factorial <- factorial * i
            Fin Para
            Escribir n, "! = ", factorial
        FinSi
    FinSi

FIN ALGORITMO

Código

```
#include <stdio.h>
int main()
{
    int n, i, factorial;    // Declaración de variables / factorial cabe solo hasta 12

    Printf("Ingrese un entero entre 0 y 12: "); // Pide el dato
    scanf("%d", &n);        // Lee el valor

    if (n<0),               // Factorial no existen negativos
    {
        Printf("El factorial es nuesta definido para negativos\n");
    }
    else if (n>12)          // ¡¡! No cabe en un int, se desborda
    {
        printf("Solo se calcula hasta 12!\n"); // Aviso del límite
    }
    else                    // 0<=n <=12
    {
        factorial = 1;
        for (i=1; i<=n; i++)
        {
            factorial= factorial *i;  // Acumulador Multiplicativo
        }
        printf("%d! = %d\n", n, factorial); // Muestra el resultado
    }
}
```

1.11 Serie de Fibonacci

Seudocódigo

```
ALGORITMO Serie_Fibonacci
    Definir n, i, anterior, actual, siguiente Como Entero
    Escribir "Ingrese cuantos términos desea (n >= 1): "
    Leer n
    Si n<1 Entonces
        Escribir "n debe ser mayor o igual que 1"
    Sino
        anterior <- 0
        actual <- 1
        Para i <- 1 Hasta n Con Paso 1 Hacer
            Escribir sin Saltar anterior, " "
            Siguiente <- anterior + actual
            anterior <- actual
            actual <- siguiente
        Fin Para
    FinSi

FIN ALGORITMO
```

Código

```
#include <stdio.h>
int main ()
{
    int n, i, anterior, actual, siguiente ; // Declaración de variables

    Printf("Cuantos términos desea (n >= 1): "); // Pide dato
    >canf ("%d", &n);                          // Lee el dato

    if (n>0)                    // validar se necesita al menos un término
    {
        Printf("n debe ser mayor o igual que 1\n"); // aviso
    }
    else
    {
        anterior = 0;           // Primer término de la serie
        actual = 1;             // segundo término
        for (i=1; i<=n; i++)    // Una vuelta por cada término que se muestra.
        {
            Printf("%d ", anterior); // Muestra antes de actualizar: 1ero el 0
            Siguiente = anterior + actual; // nuevo término: suma de los dos anteriores
            anterior = actual;      // desplaza el par
            actual = Siguiente;     // termina el desplazamiento
        }
        Printf("\n");           // Cierra la línea
    }
    return 0                    // fin
}
```

1.12 Suma de Dígitos

Seudocódigo

ALGORITMO SumaDeDigitos

    Definir n, Suma Como Entero
    Escribir "Ingrese un entero positivo: "
    Leer n
    Si n<=0 Entonces
        Escribir "El número debe ser positivo"
    SiNo
        Suma <- 0
        Mientras n >0 Hacer
            Suma <- Suma + (n MOD 10)
            n <- trunc (n/10)
        FinMientras
        Escribir "La suma de los dígitos es: ", suma
    FinSi

FIN ALGORITMO

Código

```
#include <stdio.h>
int main ()
{
    int n, suma;                    // Suma = acumulador / valores enteros

    printf("Ingrese un entero positivo: "); // Pide el dato
    scanf("%d", &n);                // Lee el dato

    if (n<=0)                       // Validar el número ingresado
    {
        Printf("El numero debe ser positivo\n"); // Aviso
    }
    else
    {
        suma =0;                    // El acumulador empieza en 0
        while (n >0)                // Vuelta por cada dígito
        {
            suma =suma +(n%10); // n %10 = ultimo digito; se acumula
            n = n /10;              //int / int corta decimales: elimina el último dígito
        }
        printf("La suma de los digitos es: %d\n", suma); // Resultado
    }
    return 0                        // Fin Correcto
}
```

1.13 Numero Capicua

Seudocódigo

ALGORITMO Numero_Capicua

    Definir n, copia, invertido Como Entero;
    Escribir "Ingrese un entero positivo:"
    Si n <= 0 Entonces
        Escribir "El número debe ser positivo"
    SINO
        copia <- n
        invertido <- 0
        Mientras copia > 0 Hacer
            invertido <- invertido * 10 + (copia MOD 10)
            copia <- trunc(copia / 10)
        Fin Mientras
        Si invertido = n Entonces
            Escribir n, " es capicúa"
        Si NO
            Escribir n, " no es capicúa"
        Fin Si
    Fin Si

FIN ALGORITMO

Código

```
#include <stdio.h>
int main()
{
    int n, copia, invertido;

    Printf("Ingrese un entero positivo: ");  // Pide dato
    scanf("%d", &n);                         // Lee el dato

    if (n <= 0)                              // Valida pidiendo un positivo
    {
        Printf("El numero debe ser positivo\n");
    }
    else
    {
        copia = n;                           // Copia n y lo deja intacto para poder comparar
        invertido = 0;                       // El numero al reves empieza por 0

        while (copia > 0)                    // Una vuelta por cada dígito
        {
            invertido = invertido * 10 + (copia % 10); // *10 desplaza los dígitos
            copia = copia / 10;              // elimina el último dígito de la copia.
        }
```

UNIVE

```
    if (inverso == n)          // Un capicua es = a su inverso
        printf("%d es capicua\n", n);
    else
        printf("%d no es capicua\n", n);
}
return 0;                      // Fin Correcto
}
```

1,14 Máximo común divisor

Seudocódigo

ALGORITMO MCD (EUCLIDES)

Definir a,b, resto Como Entero
Escribir "Ingrese el primer entero positivo: "
Leer a
Escribir "Ingrese el segundo entero positivo: "
Leer b
Si a<=0 ó b<=0 Entonces
    Escribir "Ambos números deben ser positivos"
Sino
    Mientras b<>0 Hacer
        resto <- a Mod b
        a <- b
        b <- resto
    Fin Mientras
    Escribir "El máximo común divisor es: ", a
Fin Si

FIN ALGORITMO

Código

```
#include <stdio.h>
int main ()
{
   int a, b, resto;  // Declaración de variables / resto necesario

PrintfC"Primer entero positivo: ");  // Pide Primer dato
scanf("%d", &a);                     // Lee el dato a
Printf("Segundo entero positivo: ")  // Pide Segundo dato
scanf("%d", &b);                     // Lee el dato b

if (a <=0 || b<=0)  // ||=0: basta que uno no sea positivo para rechazar
{
   PrintfC"Ambos números deben ser positivos");
}
 else
{
```

ESTILO

```
while (b!=0)          // Euclides se detiene cuado el resto es 0
{
    resto = a%b;  // resto de dividir a entre b
    a=b;          // el divisor pasa a ser el nuevo dividendo
    b= resto;     // el resto pasa a ser el nuevo divisor
}
printf("El maximo comun divisor es: %d\n", a); // "a" contiene MCD
}
return 0;                    // Fin Correcto

}
```

1.13 Año bisiesto

Seudocódigo

```
ALGORITMO  AnioBisiesto

    Definir A,B Como Entero
    Escribir "Ingrese un año: "
    Leer A,B
    Si A,B <=0 Entonces
        Escribir "El año debe ser positivo"
    SiNo
        Si (A,B MOD 4=0 Y A,B Mod 100 <>0) O A,B MOD 400=0
        Entonces
            Escribir A,B," es bisiesto"
        SiNo
            Escribir A,B," no es bisiesto"
        FinSi
    FinSi

FIN ALGORITMO
```

Código

```
#include <stdio.h>
int main ()
{
    int A,B; // Año definido como entero

Printf("Ingrese un anio: "); // Pide un dato
scanf("%d", &A,B);          // Lee el dato

if (A,B <=0)                // Validando con un valor positivo
{
    printf("El anio debe ser positivo\n"); // Aviso
}
else
{
    if ((A,B %4==0 && A,B %100 !=0) || A,B %400 ==0) // Multiplo de
                                                     4 y no de 100
```

```
        Printf("%d es bisesto\n", AnB); // los parentesis dejan clara la regla
    else
        Printf("%d no es bisesto\n", AnB); // igual que aqui; igual en el otro caso
}
return 0;                   // Fin Correcto
}
```

1.16 Número Perfecto

Seudocódigo

```
ALGORITMO NúmeroPerfecto

    Definir n, d, suma Como Entero
    Escribir "Ingrese un entero positivo: "
    Leer n
    Si n<=0 Entonces
        Escribir "El número debe ser positivo"
    SiNo
        suma <- 0
        Para d <- 1 Hasta n-1 Con Paso 1 Hacer
            Si n Mod d = 0 Entonces
                suma <- suma + d
            FinSi
        FinPara
        Si Suma = n Entonces
            Escribir n, " es perfecto"
        SiNo
            Escribir n, " no es perfecto"
        FinSi
    FinSi

FINALGORITMO
```

Código

```
# Include <stdio.h>
int main()
{
    int n, d, suma;             // Definimos variables

    Printf("Ingrese un entero positivo: "); // Pide dato
    Scanf("%d", &n);                        // Lee el dato

    if (n <=0)                  // Validar el dato: debe ser positivo
    {
      Printf("El número debe ser positivo\n"); // Aviso
    }
    else
    {
       Suma = 0;                // Acumulador empieza en 0
```

ESTILO

```
for (d = 1; d <= n-1; d++) // Divisores propios: los menores que n
{
    if (n % d == 0)        // ¿d divide exactamente a n?
        suma = suma + d;   // Entonces se acumula
}
if (suma == n)             // Perfecto = igual a la suma de sus divisores propios
    printf("%d es perfecto\n", n);
else
    printf("%d no es perfecto\n", n);
}
return 0;                  // Fin correcto
```

3

1.17 Conversión de Decimal a Binario

Seudocodigo

```
ALGORITMO DECIMAL_A_BINARIO

    Definir n, binario, potencia Como Entero
    Escribir "Ingrese un entero entre 1 y 1023: "
    Leer n
    Si n <= 0 Entonces
        Escribir "El número debe ser positivo"
    SiNo
        Si n > 1023 Entonces
            Escribir "Solo se convierte hasta 1023"
        SiNo
            binario <- 0
            potencia <- 1
            Mientras n > 0 Hacer
                binario <- binario + (n MOD 2) * potencia
                n <- trunc(n/2)
                Si n > 0 Entonces
                    potencia <- potencia * 10
                FinSi
            Fin Mientras
            Escribir "La respuesta binaria es: ", binario
        FinSi
    FinSi

FIN ALGORITMO
```

Código

```
#include <stdio.h>
int main()
{
    int n, binario, potencia;   // Definición de variables
```

```
Printf("Ingrese un entero entre 1 y 1023: "); // Pide el dato entre 1 y 1023
scanf("%d", &n);                              // Lee el dato

if (n<=0)                                     // Condicion: el valor debe ser positivo
{
   Printf("El número debe ser positivo\n");   // Aviso
}
else if (n>1023)                              // Un int solo admite 10 dígitos (1111111111 = 1023)
{
   Printf("Solo se admite hasta 1023\n");     // Aviso del límite
}
else
{
   binario=0;                                 // resultado empieza en cero
   Potencia=1;                                // Posición del dígito

   While (n>0)                                // Una vuelta por cada dígito binario
   {
      binario=binario+(n%2)*Potencia;         // Operación potencia
      n=n/2;                                  // Pasa al siguiente dígito dividiendo entre 2
      If(n>0)                                 // Evita multiplicar de más en la ultima vuelta
         Potencia=Potencia*10;                // Siguiente posición hacia la izquierda
   }
   Printf("La representación binaria es: %d\n", binario); // Resultado
}
return 0;                                     // Fin Correcto
}
```

1.13 Triángulo de Asterisco

```
ALGORITMO Triángulo De Asterisco

   Definir n, i, j Como Entero
   Escribir "Ingrese el número de filas (n>=1)"
   Leer n
   Si n<1 Entonces
      Escribir "n debe ser mayor o igual a 1"
   Sino
      Para i<-1 Hasta n Con Paso 1 Hacer
         Escribir Sin Saltar "*"
      FinPara
      Escribir ""
   FinSi

FIN ALGORITMO
```

CÓDIGO

```
#include <stdio.h>
int main()
{
   int n,i,j                                  // Definición como enteros
```

```
Printf ("Ingrese el número de filas (n>=1): "); // Pide dato
Iconf ("%d", &n);                                // Lee dato

if (n < 1)                                       // Validar: necesita al menos una fila
{
   Printf("n debe ser mayor o igual que 1\n");   // Aviso
}
else
{
      for (i=1; i<=n; i++)                       // ciclo externo
      {
         for (j = 1; j <= i ; j++)               // ciclo interno
         {
            Printf("*");                        // Sin \n: los asteriscos quedan en la misma línea
         }
         printf("\n");                          // Salto de línea una vez por fila
      }
   }
   return 0;                                     // Fin Correcto
}
```

1.19 Segundo Mayor de una Secuencia

Seudocodigo

```
ALGORITMO SEGUNDO MAYOR

Definir x, mayor, segundo Como Entero
mayor <- 0
segundo <- 0
Escribir "Ingrese un valor positivo (0 termina):"
Leer x
Mientras x <> 0 Hacer
   Si x > mayor Entonces
      segundo <- mayor
      mayor <- x
   SiNo
      Si x > segundo Y x < mayor Entonces
         segundo <- x
      FinSi
   FinSi
   Escribir "Ingrese otro valor positivo (0 termina):"
   Leer x
FINMIENTRAS
Si segundo = 0 Entonces
   Escribir "Se necesitan al menos dos valores distintos"
SiNo
   Escribir "Mayor = ", mayor, " Segundo Mayor = ", segundo
FinSi

Fin ALGORITMO
```

UNIVERSID

Código

```
#include <stdio.h>
int main ()
{
  int x, mayor=0, segundo=0; // Definir variables
  printf("Entero positivo (0 termina): "); // Pide un dato
  scanf("%d", &x);                  // Lee ese dato

  while (x!=0)   // el 0 es centinela, termina y no cuenta el dato
  {
    if (x>mayor)    // ¿Nuevo máximo?
    {
      segundo = mayor; // Máximo anterior pasa a segundo
      mayor = x;       // x es el nuevo máximo
    }
    else if (x>segundo && x<mayor) // Si no es máximo: ¿supera al segundo?
    {
      segundo = x;          // x es el nuevo segundo mayor
    }
    Printf("otro entero positivo (0 termina): "); // Pide el siguiente
    scanf("%d", &x);          // relectura: sigue no termina.
  }
  if (segundo==0)      // Si segundo sigue en 0, hubo menos de dos valores distintos.
      Printf("Se necesitan al menos dos valores distintos\n");
  else
      printf("Mayor = %d, segundo mayor = %d\n", mayor, segundo); // Muestra ambos
  return 0;          // Fin Check
}
```

1.25 Adivina el número

ALGORITMO Adivina el Número

Definir secreto, intento, intentos Como Entero
secreto <- 42
intentos <- 0
Repetir
    Escribir "Adivina el número (entre 1 y 100): "
    Leer intento
    intentos <- intentos + 1
    Si intento < secreto entonces
        Escribir "Más alto"
    SiNo
        Si intento > secreto Entonces
            Escribir "Más bajo"
        SiNo
            Escribir "¡Acertaste!"
        FinSi
    Hasta que intento = secreto
    Escribir "Intentos usados: ", intentos
FIN ALGORITMO

Código

```
#Include <studio.h>
int main ( )
{
   Const int Secreto = 42;     // Declarar un valor fijo
   int intento, intentos = 0;  // Declarar variables enteras que empiezan en 0

   do                          // Bloque se ejecute siempre al menos 1 vez
   {
      Printf("Adivina el número (entre 1 y 100): ");  // Pide un número
      Scanf("%d", &intento);   // Lo lee (intento primer valor)
      intentos++;

      if (intento < secreto)          // Menor que el secreto
         Printf("Mas alto\n");
      else if (intento > secreto)     // Mayor que el secreto
         printf("Mas bajo\n");
      else                            // Igual
         Printf("Acertaste!\n");      // Acierto
   }
   While (intento != secreto);  // Repite Mientras no acierte

   Printf("Intentos usados: %d\n", intentos);  // Muestra el total
   return 0;
}                               // Fin Correcto
```